

第 6 章-深度神经网络 – 第 6.2 节-多层网络

《深度学习基础》课程小组

2026 年 5 月 3 日

目录

1	6.2.1 Parameter matrices	3
2	6.2.2 Universal approximation	3
3	6.2.3 Hidden unit activation functions	4
4	6.2.4 Weight-space symmetries	8

In the previous section, we saw that to apply linear models of the form (6.1) to problems involving large-scale data sets and high-dimensional spaces, we need to find a set of basis functions that is tuned to the problem being solved.

The key idea behind neural networks is to choose basis functions $\phi_j(\mathbf{x})$ that themselves have learnable parameters and then allow these parameters to be adjusted, along with the coefficients $\{w_j\}$, during training.

We then optimize the whole model by minimizing an error function using gradient-based optimization methods, such as stochastic gradient descent, where the error function is defined jointly across all the parameters in the model.

There are, of course, many ways to construct parametric nonlinear basis functions.

One key requirement is that they must be differentiable functions of their learnable parameters so that we can apply gradient-based optimization.

The most successful choice has been to use basis functions that follow the same form as (6.1), so that each basis function is itself a nonlinear function of a linear combination of the inputs, where the coefficients in the linear combination are learnable parameters.

Note that this construction can clearly be extended recursively to give a hierarchical model with many layers, which forms the basis for deep neural networks.

Consider a basic neural network model having two layers of learnable parameters.

First, we construct M linear combinations of the input variables $x_1, \dots, x_D$ in the form

$$a_j^{(1)} = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \quad (6.7)$$

where $j = 1, \dots, M$, and the superscript (1) indicates that the corresponding parameters are in the first 'layer' of the network.

We will refer to the parameters $w_{ji}^{(1)}$ as weights and the parameters $w_{j0}^{(1)}$ as biases, while the quantities $a_j^{(1)}$ are called pre-activations.

Each of the quantities a_j is then transformed using a differentiable, nonlinear activation function $h(\cdot)$ to give

$$z_j^{(1)} = h(a_j^{(1)}), \quad (6.8)$$

which represent the outputs of the basis functions in (6.1).

In the context of neural networks, these basis functions are called hidden units.

We will explore various choices for the nonlinear function $h(\cdot)$ shortly, but here we note that provided the derivative $h'(\cdot)$ can be evaluated, then the overall network function will be differentiable.

Following (6.1), these values are again linearly combined to give

$$a_k^{(2)} = \sum_{j=1}^M w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)} \quad (6.9)$$

where $k = 1, \dots, K$, and K is the total number of outputs.

This transformation corresponds to the second layer of the network, and again the $w_{kj}^{(2)}$ are bias parameters.

Finally, the $\{a_k^{(2)}\}$ are transformed using an appropriate output-unit activation function $f(\cdot)$ to give a set of network outputs y_k .

A two-layer neural network can be represented in diagram form as shown in Figure 6.9.

1 6.2.1 Parameter matrices

As we discussed in the context of linear regression models, the bias parameters in (6.7) can be absorbed into the set of weight parameters by defining an additional input variable x_0 whose value is clamped at $x_0 = 1$, so that (6.7) takes the form

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i. \quad (6.10)$$

We can similarly absorb the second-layer biases into the second-layer weights, so that the overall network function becomes

$$y_k(\mathbf{x}, \mathbf{w}) = f \left(\sum_{j=0}^M w_{kj}^{(2)} h \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right). \quad (6.11)$$

Another notation that will prove convenient at various points in the book is to represent the inputs as a column vector $\mathbf{x} = (x_1, \dots, x_N)^T$ and then to gather the weight and bias parameters in (6.11) into matrices to give

$$\mathbf{y}(\mathbf{x}, \mathbf{w}) = f \left(\mathbf{W}^{(2)} h \left(\mathbf{W}^{(1)} \mathbf{x} \right) \right) \quad (6.12)$$

where $f(\cdot)$ and $h(\cdot)$ are evaluated on each vector element separately.

2 6.2.2 Universal approximation

The capability of a two-layer network to model a broad range of functions is illustrated in Figure 6.10.

This figure also shows how individual hidden units work collaboratively to approximate the final function.

The role of hidden units in a simple classification problem is illustrated in Figure 6.11.

练习：复现图 6-11，使用两层神经网络（两个输入节点、两个隐藏节点（tanh 激活）、一个输出节点（sigmoid 激活）进行简单二分类。

The approximation properties of two-layer feed-forward networks were widely studied in the 1980s, with various theorems showing that, for a wide range of activation functions, such networks can approximate any function defined over a continuous subset of $\mathbb{R}^D$ to arbitrary accuracy (Funahashi, 1989; Cybenko, 1989; Hornik, Stinchcombe, and White, 1989; Leshno et al., 1993).

A similar result holds for functions from any finite-dimensional discrete space to any another.

Neural networks are therefore said to be universal approximators.

Although such theorems are reassuring, they tell us only that there exists a network that can represent the required function.

In some cases, they may require networks that have an exponentially large number of hidden units.

Moreover, they say nothing about whether such a network can be found by a learning algorithm.

Furthermore, we will see later that the no free lunch theorem says that we can never find a truly universal machine learning algorithm.

Finally, although networks having two layers of weights are universal approximators, in a practical application, there can be huge benefits in considering networks having many more than two layers that can learn hierarchical internal representations.

All these points support the drive towards deep learning.

3 6.2.3 Hidden unit activation functions

We have seen that the activation functions for the output units are determined by the kind of distribution being modelled.

For the hidden units, however, the only requirement is that they need to be differentiable, which leaves a wide range of possibilities.

In most cases, all the hidden units in a network will be given the same activation function, although in principle there is no reason why different choices could not be applied in different parts of the network.

The simplest option for a hidden unit activation function is the identity function, which means that all the hidden units become linear.

However, for any such network, we can always find an equivalent network without hidden units.

This follows from the fact that the composition of successive linear transformations is itself a linear transformation, and so its representational capability is no greater than that of a single linear layer.

However, if the number of hidden units is smaller than either the number of input or output units, then the transformations that such a network can generate are not the most general possible linear transformation from inputs to outputs because information is lost in the dimensionality reduction at the hidden units.

Consider a network with N inputs, M hidden units, and K outputs, and where all activation functions are linear.

Such a network has $M(N+K)$ parameters, whereas a linear transformation of inputs directly to outputs would have NK parameters.

If M is small relative to N or K , or both, this leads to a two-layer linear network having fewer parameters than the direct linear mapping, corresponding to a rank-deficient transformation.

Such 'bottleneck' networks of linear units corresponds to a standard data analysis technique called principal component analysis.

主成分分析可以看作是一个不带激活函数的两层神经网络。

In general, however, there is limited interest in using multilayer networks of linear units since the overall function computed by such a network is still linear.

A simple, nonlinear differentiable function is the logistic sigmoid given by

$$\sigma(a) = \frac{1}{1 + \exp(-a)}, \quad (6.13)$$

which is plotted in Figure 5.12.

This was widely used in the early years of work on multilayer neural networks and was partly inspired by studies of the properties of biological neurons.

A closely related function is $\tanh$, which is defined by

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}, \quad (6.14)$$

which is plotted in Figure 6.12(a).

This function differs from the logistic sigmoid by a linear transformation of its input and its output values, and so for any network with logistic-sigmoid hidden-unit activation functions, there is an equivalent network with $\tanh$ activation functions.

练习：在 $y = \frac{1}{1+e^{-x}}$ 与 $u = \frac{e^v - e^{-v}}{e^v + e^{-v}}$ 之间通过可逆线性变换建立联系。

However, when training a network, these are not necessarily equivalent because for gradient-based optimization, the network weights and biases need to be initialized, and so if the activation functions are changed, then the initialization scheme must be adjusted accordingly.

A 'hard' version of the tanh function (Collobert, 2004) is given by

$$h(a) = \max(-1, \min(1, a)) \quad (6.15)$$

and is plotted in Figure 6.12(b).

A major drawback of both the logistic sigmoid and the tanh activation functions is that the gradients go to zero exponentially when the inputs have either large positive or large negative values.

We will discuss this 'vanishing gradients' issue later, but for the moment, we note that it will generally be better to use activation functions with non-zero gradients, at least when the input takes a large positive value.

One such choice is the softplus activation function given by

$$h(a) = \ln(1 + \exp(a)), \quad (6.16)$$

which is plotted in Figure 6.12(c).

For $a \gg 1$, we have $h(a) \simeq a$, and so the gradient remains non-zero even when the input to the activation function is large and positive, thereby helping to alleviate the vanishing gradients problem.

An even simpler choice of activation function is the rectified linear unit or ReLU, which is defined by

$$h(a) = \max(0, a) \quad (6.17)$$

and which is plotted in Figure 6.12(d).

Empirically, this is one of the best-performing activation functions, and it is in widespread use.

Note that strictly speaking, the derivative of the ReLU function is not defined when $a = 0$, but in practice this can be safely ignored.

The softplus function (6.16) can be viewed as a smoothed version of the ReLU and is therefore also sometimes called soft ReLU.

Although the ReLU has a non-zero gradient for positive input values, this is not the case for negative inputs, which can mean that some hidden units receive no 'error signal' during training.

A modification of ReLU that seeks to avoid this issue is called a leaky ReLU and is defined by

$$h(a) = \max(0, a) + \alpha \min(0, a), \quad (6.18)$$

where $0 < \alpha < 1$.

This function is plotted in Figure 6.12(e).

Unlike ReLU, this has a nonzero gradient for input values $a < 0$, which ensures that there is a signal to drive training.

A variant of this activation function uses $\alpha = -1$, in which case $h(a) = |a|$, which is plotted in Figure 6.12(f).

Another variant allows each hidden unit to have its own value α_j , which can be learned during network training by evaluating gradients with respect to the $\{\alpha_j\}$ along with the gradients with respect to the weights and biases.

The introduction of ReLU gave a big improvement in training efficiency over previous sigmoidal activation functions (Krizhevsky, Sutskever, and Hinton, 2012).

As well as allowing deeper networks to be trained efficiently, it is much less sensitive to the random initialization of the weights.

It is also well suited to a low-precision implementation, such as 8-bit fixed versus 64-bit floating point, and it is computationally cheap to evaluate.

Many practical applications simply use ReLU units as the default unless the goal is explicitly to explore the effects of different choices of activation function.

4 6.2.4 Weight-space symmetries

One property of feed-forward networks is that multiple distinct choices for the weight vector $\mathbf{w}$ can all give rise to the same mapping function from inputs to outputs (Chen, Lu, and Hecht-Nielsen, 1993).

Consider a two-layer network of the form shown in Figure 6.9 with M hidden units having tanh activation functions and full connectivity in both layers.

If we change the sign of all the weights and the bias feeding into a particular hidden unit, then, for a given input data point, the sign of the pre-activation of the hidden unit will be reversed, and therefore so too will the activation, because tanh is an odd function, so that $\tanh(-a) = -\tanh(a)$.

This transformation can be exactly compensated for by changing the sign of all the weights leading out of that hidden unit.

Thus, by changing the signs of a particular group of weights (and a bias), the input-output mapping function represented by the network is unchanged, and so we have found two different weight vectors that give rise to the same mapping function.

For M hidden units, there will be M such 'sign-flip' symmetries, and thus, any given weight vector will be one of a set 2^M equivalent weight vectors.

Similarly, imagine that we interchange the values of all of the weights (and the bias) leading both into and out of a particular hidden unit with the corresponding values of the weights (and bias) associated with a different hidden unit.

Again, this clearly leaves the network input-output mapping function unchanged, but it corresponds to a different choice of weight vector.

For M hidden units, any given weight vector will belong to a set of $M \times (M - 1) \times \dots \times 2 \times 1 = M!$ equivalent weight vectors associated with this interchange symmetry, corresponding to the $M!$ different orderings of the hidden units.

The network will therefore have an overall weight-space symmetry factor of $M!2^M$.

For networks with more than two layers of weights, the total level of symmetry will be given by the product of such factors, one for each layer of hidden units.

It turns out that these factors account for all the symmetries in weight space (except for possible accidental symmetries due to specific choices for the weight values).

Furthermore, the existence of these symmetries is not a particular property of the tanh function but applies to a wide range of activation functions (Kurková and Kainen, 1994).

In general, these symmetries in weight space are of little practical consequence, since network training

aims to find a specific setting for the parameters, and the existence of other, equivalent, settings is of little consequence.

However, weight-space symmetries do play a role when Bayesian methods are used to evaluate the probability distribution over networks of different sizes (Bishop, 2006).